

Projekt: Software Engineering

Anforderungsdokument

MietManager

Portfolio von Michael Bangas

Matrikelnummer: IU14132885

Kursbezeichnung: Projekt: Software Engineering (DLMCSPSE01_D)

Studiengang: M.Sc. Informatik

IU Internationale Hochschule

25. September 2026

Inhaltsverzeichnis

1	Stakeholder	1
1.1	Vermieter (primärer Nutzer)	1
1.2	Mieter (indirekter Stakeholder)	1
1.3	Tutoren (im Rahmen des Kurses)	1
2	Funktionale Anforderungen	1
2.1	User Stories	1
2.2	UML-Use-Case-Diagramm	3
3	Nichtfunktionale Anforderungen	4
4	Glossar	5
5	Fazit	5
	Literaturverzeichnis	6

Abbildungsverzeichnis

2.1 UML-Use-Case-Diagramm 4

Tabellenverzeichnis

2.1 User Stories mit Akzeptanzkriterien 2

4.1 Glossar 5

1 Stakeholder

Dieser Abschnitt identifiziert die am Projekt beteiligten Ziel- und Benutzergruppen und beschreibt ihre jeweilige Rolle sowie ihr Interesse an der Anwendung.

1.1 Vermieter (primärer Nutzer)

Private Kleinvermieter mit wenigen Wohneinheiten sind die primäre Zielgruppe von MietManager. Sie bedienen die Anwendung direkt, verwalten ihre Immobilien, Mieter und Mietverträge und erstellen die Nebenkostenabrechnung. Diese Gruppe verwaltet ihre Objekte bislang häufig mit Excel oder auf Papier und hat in der Regel keine spezialisierte Software im Einsatz.

1.2 Mieter (indirekter Stakeholder)

Mieter interagieren nicht direkt mit der Anwendung, sind aber indirekt betroffen. Sie erhalten die von MietManager erzeugte Nebenkostenabrechnung als personalisierten Geschäftsbrief. Die Verständlichkeit und Korrektheit der Abrechnung wirkt sich damit unmittelbar auf diese Gruppe aus, auch wenn sie kein Benutzerkonto oder keinen Zugriff auf die Software besitzt.

1.3 Tutoren (im Rahmen des Kurses)

Im Rahmen dieses Portfoliokurses sind die betreuenden Tutoren gemäß Aufgabenstellung als Zielgruppe bzw. Benutzer der zu erstellenden Anwendung zu betrachten. Sie bewerten die Anwendung aus Sicht eines potenziellen Vermieters und geben Feedback zu Bedienbarkeit und Funktionsumfang.

2 Funktionale Anforderungen

Dieser Abschnitt beschreibt die funktionalen Anforderungen an MietManager zunächst als priorisierte User Stories und visualisiert sie anschließend in einem UML-Use-Case-Diagramm.

2.1 User Stories

Tabelle 2.1 listet die funktionalen Anforderungen als User Stories im Format „Als <Rolle> möchte ich <Ziel>, um <Nutzen> zu erreichen.“ auf, ordnet jeder ein prüfbares Akzeptanzkriterium zu und priorisiert sie nach der MoSCoW-Methode.

Tabelle 2.1: User Stories mit Akzeptanzkriterien

ID	User Story	Akzeptanzkriterium	Priorität
FA-01	Als Vermieter möchte ich Immobilien und deren Mieteinheiten anlegen und pflegen, um meinen Immobilienbestand zentral zu verwalten.	Nach dem Anlegen einer Immobilie mit Bezeichnung und Adresse sowie mindestens einer Mieteinheit erscheinen beide in der Immobilienübersicht.	Muss
FA-02	Als Vermieter möchte ich Mieter mit ihren Kontaktdaten erfassen, um alle mieterbezogenen Informationen an einem Ort zu haben.	Nach der Eingabe von Name und Kontaktdaten erscheint der Mieter in der Mieterliste und kann bei der Anlage eines Mietvertrags ausgewählt werden.	Muss
FA-03	Als Vermieter möchte ich Mietverträge anlegen, die automatisch auf zeitliche Überschneidungen je Mieteinheit geprüft werden, um Doppelvermietungen zu vermeiden.	Beim Speichern eines Mietvertrags, dessen Zeitraum sich mit einem bestehenden Mietvertrag derselben Mieteinheit überschneidet, wird eine Fehlermeldung angezeigt und der Vertrag nicht gespeichert.	Muss
FA-04	Als Vermieter möchte ich Kostenpositionen je Immobilie und Zeitraum erfassen, um die Grundlage für die Nebenkostenabrechnung zu schaffen.	Nach der Eingabe von Kostenart, Betrag, Zeitraum und Umlageschlüssel erscheint die Kostenposition in der Liste der Immobilie.	Muss
FA-05	Als Vermieter möchte ich die Nebenkostenabrechnung automatisch nach einem frei wählbaren Umlageschlüssel berechnen lassen, um Zeit zu sparen und Rechenfehler zu vermeiden.	Beim Erstellen einer Abrechnung werden alle im gewählten Zeitraum aktiven Kostenpositionen gemäß ihrem Umlageschlüssel auf die aktiven Mietverträge verteilt. Die Summe der berechneten Kostenanteile entspricht dabei der Summe der zugrunde liegenden Kostenpositionen.	Muss
FA-06	Als Vermieter möchte ich die berechnete Nebenkostenabrechnung als versandfertigen, personalisierten PDF-Brief exportieren, um sie ohne Zwischenschritt an meine Mieter zu verschicken.	Nach Klick auf einen „Exportieren-Button“ wird je Mietvertrag eine PDF-Datei mit Briefkopf, persönlicher Anrede, Kostenaufschlüsselung, Saldo und Grußformel im gewählten Zielordner erzeugt.	Muss

Tabelle 2.1 – Fortsetzung

ID	User Story	Akzeptanzkriterium	Priorität
FA-07	Als Vermieter möchte ich meine eigenen Stammdaten einmalig in einem Profil hinterlegen, damit sie automatisch als Briefkopf verwendet werden.	Nach dem Ausfüllen und Speichern von Name, Straße, PLZ und Ort im Vermieterprofil erscheinen diese Daten als Absender im Briefkopf jeder neu exportierten Nebenkostenabrechnung. Bei unvollständigem Profil wird der Export mit einer Fehlermeldung verhindert.	Muss
FA-08	Als Vermieter möchte ich ein Dashboard mit den wichtigsten Kennzahlen sehen, um schnell einen Überblick über Leerstand und Mieteinnahmen zu erhalten.	Nach dem Anlegen oder Ändern einer Immobilie, Mieteinheit oder eines Mietvertrags zeigt das Dashboard beim nächsten Öffnen aktualisierte Kennzahlen zu Immobilien, Einheiten, Leerstand, aktiven Verträgen und Mieteinnahmen.	Soll

2.2 UML-Use-Case-Diagramm

Abbildung 2.1 visualisiert die funktionalen Anforderungen aus Sicht des Akteurs „Vermieter“. Da Mieter nicht direkt mit der Anwendung interagieren, treten sie im Use-Case-Diagramm nicht als Akteur auf (siehe Abschnitt 1.2). Der Anwendungsfall „Nebenkostenabrechnung erstellen“ entspricht User Story FA-05 und wird im Spezifikationsdokument auf Ebene der einzelnen Verarbeitungsschritte als UML-Aktivitätsdiagramm detailliert dargestellt. Die dabei verarbeiteten Geschäftsobjekte sind im zugehörigen UML-Klassendiagramm modelliert.

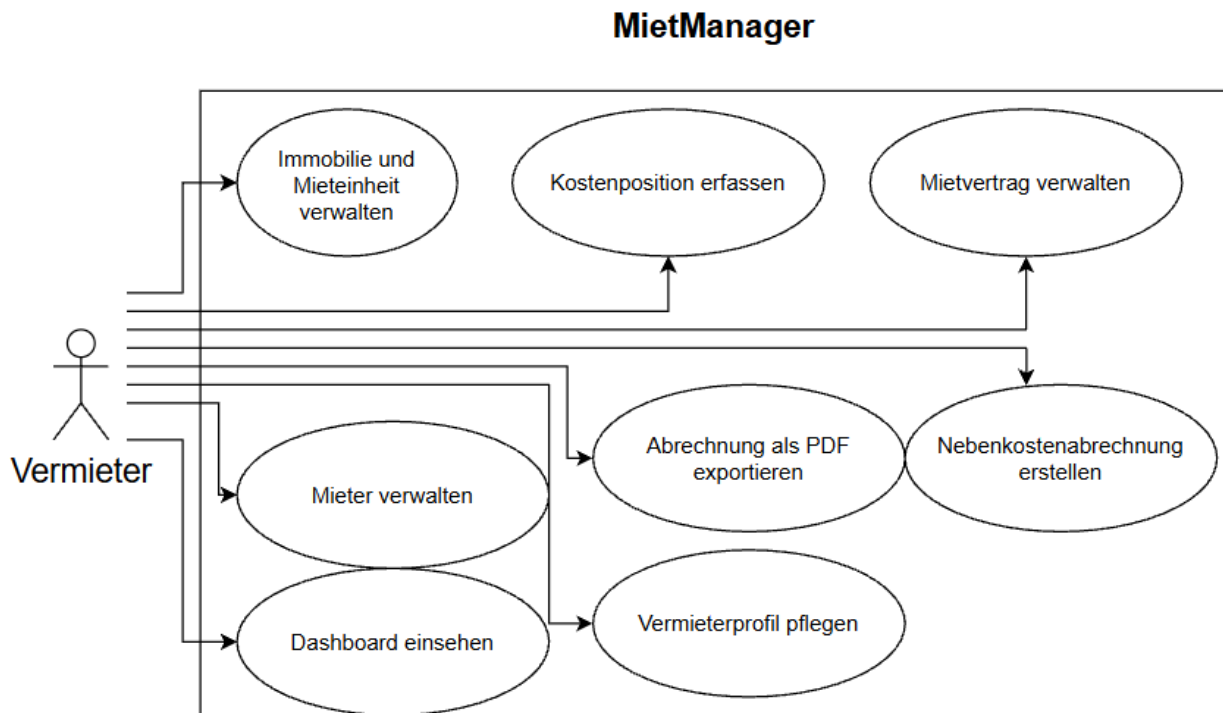


Abbildung 2.1: UML-Use-Case-Diagramm

3 Nichtfunktionale Anforderungen

Im Hinblick auf die Benutzerfreundlichkeit lässt sich der Kernprozess, das Erstellen einer Nebenkostenabrechnung, ohne Handbuch oder Einweisung in maximal drei Bedienschritten abschließen: Immobilie und Zeitraum auswählen, die Abrechnung erstellen und sie als PDF exportieren. Jeder Erfassungsdialog benennt zudem beim Speichern fehlende Pflichtfelder namentlich in einer Fehlermeldung, sodass Nutzer:innen ohne Rückfrage erkennen, was zu korrigieren ist. Beide Kriterien sind über die Testfälle TF-02, TF-10, TF-15, TF-21, TF-22, TF-23 und TF-25 des Testdokuments nachprüfbar. Bezüglich der Performance reagiert die Benutzeroberfläche bei einem Bestand von bis zu 20 Mieteinheiten innerhalb von 100 ms auf Benutzerinteraktionen. Dieser Schwellwert orientiert sich am in der Usability-Forschung etablierten Richtwert, ab dem Reaktionen von Nutzern als unmittelbar und nicht als Wartezeit wahrgenommen werden (Nielsen, 1993). Der Bestand von 20 Mieteinheiten orientiert sich an der in Abschnitt 1.1 beschriebenen Zielgruppe privater Kleinvermieter mit wenigen Wohneinheiten. Ab einem zweistelligen Wohnungsbestand wird die manuelle Verwaltung mit Excel oder auf Papier gemäß der Problemstellung zunehmend unübersichtlich, sodass genau in dieser Größenordnung der Kundennutzen von MietManager ansetzt. Hinsichtlich der Skalierbarkeit ist die Anwendung auf diesen Bestand von bis zu 20 Mieteinheiten ausgelegt, was bei realistischer Nutzung über mehrere Jahre einem Gesamtbestand von bis zu 200 Mietverträgen, inklusive beendeter Verträge, und bis zu 400 Kostenpositionen in der lokalen SQLite-Datenbank entspricht. Diese Werte sind be-

wusst als großzügige Obergrenze für einen privaten Kleinvermieter gewählt, die auch bei häufigerem Mieterwechsel und mehrjähriger Nutzung nicht überschritten werden dürfte. Darüber hinausgehende Bestände sowie gleichzeitiger Mehrbenutzerzugriff werden von der dateibasierten SQLite-Architektur bewusst nicht unterstützt, da laut MoSCoW-Priorisierung in der Projektdokumentation auf Mehrbenutzerfähigkeit verzichtet wird. Bezüglich der Zuverlässigkeit läuft jede Datenbankoperation als einzelne SQLite-Transaktion. Ein Commit schreibt dabei entweder vollständig oder gar nicht in die Datenbankdatei, wodurch bereits die Transaktionsgarantie von SQLite verhindert, dass ein bei einem Fehler abgebrochener Vorgang einen inkonsistenten Zwischenstand in der Datenbank hinterlässt. Da sämtliche Daten in einer einzelnen SQLite-Datei abgelegt werden, kann der Vermieter durch einfaches Kopieren dieser Datei jederzeit eine manuelle Sicherung anlegen und so einem Datenverlust vorbeugen. Für die Datensicherheit werden die Daten ausschließlich lokal in einer SQLite-Datenbank im Benutzerprofil gespeichert. Eine Übertragung an Dritte findet nicht statt. Hinsichtlich der Portabilität ist die Anwendung unter Windows 10/11 lauffähig, da diese Versionen zum Zeitpunkt der Projekterstellung die aktuell von Microsoft unterstützten Desktop-Betriebssystemversionen sind, ohne dass über die vom Kurs zugelassenen Standard-Laufzeitumgebungen hinaus zusätzliche Software installiert werden muss.

4 Glossar

Das folgende Glossar erläutert die im Projekt verwendeten Fachbegriffe und Abkürzungen.

Begriff	Erläuterung
Immobilie	Ein vermietetes Gebäude oder Grundstück, bestehend aus einer oder mehreren Mieteinheiten.
Mieteinheit	Einzelne vermietbare Einheit innerhalb einer Immobilie.
Umlageschlüssel	Regel, nach der die Kosten einer Kostenposition auf die Mieteinheiten einer Immobilie verteilt werden z.B. nach Fläche, Personenzahl oder Anzahl Einheiten.
Kostenposition	Eine einzelne erfasste Betriebskostenart einer Immobilie für einen bestimmten Zeitraum.
Nebenkostenabrechnung	Das aus den Kostenpositionen eines Zeitraums berechnete Ergebnis je Mietvertrag.
Vermieterprofil	Die Stammdaten des Vermieters, die als Absender auf der Nebenkostenabrechnung erscheinen.
MoSCoW-Priorisierung	Methode aus dem Anforderungsmanagement, die Anforderungen in Muss (Must have), Soll (Should have), Kann (Could have) und Verzicht (Won't have) einteilt.

Tabelle 4.1: Glossar

5 Fazit

Dieses Dokument legt mit den drei Stakeholdergruppen Vermieter, Mieter und Tutoren sowie acht funktionalen Anforderungen inklusive Akzeptanzkriterien und MoSCoW-Priorisierung, visualisiert im

UML-Use-Case-Diagramm, den funktionalen Rahmen von MietManager fest. Zusammen mit den nichtfunktionalen Anforderungen zu Benutzerfreundlichkeit, Performance, Skalierbarkeit, Zuverlässigkeit, Datensicherheit und Portabilität sowie dem Glossar legt es damit die Grundlage, auf der das Spezifikationsdokument das Datenmodell und die Geschäftsprozesse im Detail konkretisiert.

Literaturverzeichnis

Nielsen, J. (1993). *Response Times: The 3 Important Limits*. <https://www.nngroup.com/articles/response-times-3-important-limits/>